

National Institute of Business Management
School of Computing and Engineering
Course work | Assement Announcement Sheet



Course Name	BSc (Hons) Computing-25.2		
Module Name	PDSA		
Batch	25.2		
Learning Outcomes Covered (Mention according to the Module Descriptor)	1. Understand and select appropriate algorithms for solving a range of problems and reason about their complexity and efficiency.		
	2. Design and implement algorithms and data structures for novel problems.		
	3. Specify and implement methods to estimate solutions to intractable problems		
Assesement CW No	1		
Assesement Mode	Individual Group	Group (if it is group mode only)	
		Group Size	Grouping Criteria
		10	Students can select the group members
Assesement Type	Practical Test Report Software Presentation VIVA MCQ		
	If other specify	All the Task in the Course Work needed to be completed by the Students	
Hand in Date Time	30-03-2026		
Hand out Date Time	30-04-2026		
Submission Details (Format and Location)	NIBM Moodle		
Plagiarism Criteria			
Assesement CW Description			

1. Create a game menu option called Minimum Cost:
 - A company has N tasks to be completed by N employees .
 - N is changing randomly from 50 to 100 in each game round
 - write a game that takes number of Tasks as Inputs & Use 2 appropriate algorithm to determine the optimal assignment of tasks to employees that minimizes the total cost. Only One task can be assigned to one Employee.
 - In Each game round, cost (in dollars) of assigning each task to each employee changing randomly from \$20 to \$200.
 - Include the Code used for unit Testing
 - Additionally, record the time taken for each algorithm in each game round in the database.

2. Create a game menu option called Snake and Ladder Game Problem
 - Consider a standard Snake and Ladder board of size $N \times N$ with cells numbered from 1 to N^2
 - You start from cell 1 and must reach cell N^2
 - Each move is decided by rolling a six-faced dice (values 1–6).
 - If you land at the base of a ladder, you must climb to its top.
 - If you land on the mouth of a snake, you must go down to its tail.
 - User Input: size of the board N (value between 6 and 12)
 - Set the number of ladders in the board as N-2
 - Set the number of snakes in the board as N-2
 - Set the ladders & snakes starting & ending location randomly for each game round.
 - Using two different appropriate algorithms find the minimum number of dice throws required to reach the last cell.
 - In each game round ask game player to enter the minimum number of dice throws required to reach the last cell by giving 3 choices
 - Allow game players to provide answers using a user interface
 - Provide user interface when players win, lose, or draw a game
 - When a game player correctly identifies an answer, save that person's name along with the correct response in the database.
 - Additionally, record in the database how long it takes to find the minimum number of dice throws using each algorithm approaches during each game round.
 - Include the Code used for unit Testing
 - Apply appropriate validations & Exception Handling

3. Create a game menu option called Traffic simulation Problem
 - Consider a simplified traffic network where intersections are nodes and directed road segments have a maximum vehicle throughput (capacity, vehicles per minute)
 - **Nodes:** A (source), B, C, D, E, F, G, H, T (sink).

Roads	capacities (vehicles/minute)
A → B	In each game ground set a random capacity between 5 and 15
A → C	In each game ground set a random capacity between 5 and 15
A → D	In each game ground set a random capacity between 5 and 15
B → E	In each game ground set a random capacity between 5 and 15
B → F	In each game ground set a random capacity between 5 and 15
C → E	In each game ground set a random capacity between 5 and 15
C → F	In each game ground set a random capacity between 5 and 15
D → F	In each game ground set a random capacity between 5 and 15
E → G	In each game ground set a random capacity between 5 and 15
E → H	In each game ground set a random capacity between 5 and 15
F → H	In each game ground set a random capacity between 5 and 15
G → T	In each game ground set a random capacity between 5 and 15
H → T	In each game ground set a random capacity between 5 and 15

- Display the traffic network graph in UI
- In each game round ask game player to enter the maximum flow.
- Using two different appropriate algorithms identifies the maximum flow from A to T.
- Allow game players to provide answers using a user interface
- Provide user interface when players win, lose, or draw a game
- When a game player correctly identifies an answer, save that person's name along with the correct response in the database.
- Additionally, record in the database how long it takes to find maximum flow using algorithms during each game round.
- Include the Code used for unit Testing
- Apply appropriate validations & Exception Handling

4. Create a game menu option called knight's tour Problem

In this Problem, there is an empty chess board, and a knight starting from any location in the board, Identify the sequence of moves of a knight on a chessboard such that the knight visits every square exactly once.

- The chessboard size can be either 8×8 or 16×16 .
- The board size is selected at the start of the game.
- In each game round knight starting position is randomly selected.
- Allow game players to provide answers using a user interface.
- Provide user interface when players win, lose, or draw a game.
- Select appropriate Application Data Structures & algorithm to determine correct answer
- Determine the sequence of moves by applying two different algorithmic approaches.
- When a game player correctly identifies an answer, save that person's name along with the correct response in the database.
- Include the Code used for unit Testing
- Apply appropriate validations & Exception Handling

5. Create a game menu option called Sixteen queens' puzzle:

- Eight chess queens must be placed on an 16×16 chessboard in such a way that no two queens threaten each other
- Using a Sequential program, find the maximum number of solutions for this and save them as answers in the Database. & Record the Time taken for it in the Database
- Using a Threaded program, find the maximum number of solutions for this and save them as answers in the Database. & Record the Time taken for it in the Database
- compare Sequential program & the Threaded program.
- Allow game players to provide answers using a user interface.
- When a game player correctly identifies an answer, save that person's name along with the correct response in the database.
- Select appropriate Application Data Structures & algorithm to determine correct answer
- If another game player provides the same right response, indicate that the solution has already been recognized. Ask them to try again until the maximum number of solutions has been achieved.
- When all the solutions have been identified by game players, the system should clear the flag that indicates solution has already been recognized. so that future game players can give same answers.
- When a game player correctly identifies an answer, save that person's name along with the correct response in the database.
- Include the Code used for unit Testing
- Apply appropriate validations & Exception Handling

DELIVERABLES: The students should submit.

- I. Software with Source Code (Git Hub Link) Note- source code cannot be changed after the deadline (submitted by the Group Leader)
- II. Database DDL((submitted by the Group Leader)
- III. Data Dump (submitted by the each Individual)
- IV. Individual Report (submitted by the each Individual)
- V. Group Report (submitted by the Group Leader)
- VI. Small Video Clip indicating the features of the games (submitted by the Group Leader)
- VII. Contribution Table (submitted by the Group Leader)

Group Leader needs to submit the completed table

Index	Name	Marks out of 10 Based on contribution (all the members needed to agree on this)	Contribution
A			
B			
C			
D			
E			

Individual Report including:**1. Minimum Cost Problem**

- i. Explain the Program Logic used to solve the problem
- ii. Analyze the Complexity of the algorithms based on the Program outputs & Program logic
- iii. Comparison of the two algorithmic approaches
- iv. Chart Containing the time Taken for each algorithm Technique when run the Game Individually for 20 Game Rounds
- v. Video Clip Link that Demonstrate 20 Game Rounds & its time taken
- vi. Database output Screenshot for Chat Content

2. Snake and Ladder Game Problem

- i. Explain the Program Logic used to solve the problem
- ii. Analyze the Complexity of the algorithms based on the Program outputs & Program logic
- iii. Comparison of the two algorithmic approaches
- iv. Chart Containing the time Taken for each algorithm Technique when run the Game Individually for 20 Game Rounds
- v. Video Clip Link that Demonstrate 20 Game Rounds & its time taken
- vi. Database output Screenshot for Chat Content

3. Traffic simulation Problem

- i. Explain the Program Logic used to solve the problem
- ii. Analyze the Complexity of the algorithms based on the Program outputs& Program logic
- iii. Comparison of the two algorithmic approaches
- iv. Chart Containing the time Taken for each algorithm Technique when run the Game Individually for 20 Game Rounds
- v. Video Clip Link that Demonstrate 20 Game Rounds & its time taken
- vi. Database output Screenshot for Chat Content

4. knight's tour Problem

- i. Explain the Program Logic used to solve the problem
- ii. Analyze the Complexity of the algorithms based on the Program outputs& Program logic
- iii. Comparison of the recursive & iterative (non-recursive) solutions
- iv. Chart Containing the time Taken for each algorithm Technique when run the Game Individually for 20 Game Rounds
- v. Video Clip Link that Demonstrate 20 Game Rounds & its time taken
- vi. Database output Screenshot for Chat Content

5. Sixteen queens' puzzle

- i. Explain the Program Logic used to solve the problem
- ii. Analyze the Complexity of the algorithms based on the Program outputs & Program logic
- iii. Compare the time taken & logic used for the Sequential program & the Threaded program used to identify maximum number of Solutions
- iv. Chart Containing the time Taken for each algorithm Technique when run the Game Individually for 20 Game Rounds
- v. Video Clip Link that Demonstrate 20 Game Rounds & its time taken
- vi. Database output Screenshot for Chat Content

Group Report including

1. Minimum Cost Problem

1.1. Functionality:

- 1.1.1.Code Segment screenshots: “N is changing randomly from 50 to 100 in each game round”
- 1.1.2.Code Segment screenshots: “cost (in dollars) of assigning each task to each employee changing randomly from \$20 to \$200.
- 1.1.3.Code Segment screenshots: “record in the database how long it takes to find the minimum number of dice throws using each algorithm approaches during each game round”
- 1.1.4.Code Segment screenshots: “two different appropriate algorithms”
- 1.1.5.Code Segment Screenshot of “save that person's name along with the correct response in the database” Requirement
- 1.1.6.Screenshot of the Normalized DB Table Structure used for this Game Option

1.2. UI:

- 1.2.1.UI screenshot when asking user to enter the inputs & answers
- 1.2.2.Explain the Validations and Exception Handling in this application.

1.3. Code Segment screenshots: unit Testing

2. Snake and Ladder Game Problem

2.1. Functionality:

- 2.1.1.Code Segment screenshots: “Set the number of ladders in the board as N-2”
- 2.1.2.Code Segment screenshots: “Set the number of snakes in the board as N-2”
- 2.1.3.Code Segment screenshots: “Set the ladders & snakes starting & ending location randomly for each game round.”
- 2.1.4.Code Segment screenshots: “record in the database how long it takes to find the minimum number of dice throws using each algorithm approaches during each game round”
- 2.1.5.Code Segment screenshots: “two different appropriate algorithms”
- 2.1.6.Code Segment Screenshot of “save that person's name along with the correct response in the database” Requirement
- 2.1.7.Screenshot of the Normalized DB Table Structure used for this Game Option

2.2. UI:

- 2.2.1.UI screenshot when asking user to enter the inputs & answers
- 2.2.2.Explain the Validations and Exception Handling in this application.

2.3. Code Segment screenshots: unit Testing

3. Traffic simulation Problem

3.1. Functionality:

- 3.1.1.Code Segment screenshots: “Display the traffic network graph in UI”
- 3.1.2.Code Segment screenshots: “In each game ground set a random capacity between 5 and 15”
- 3.1.3.Code Segment screenshots: “Using two different appropriate algorithms identifies the maximum flow from A to T”
- 3.1.4.Code Segment Screenshot of “save that person's name along with the correct response in the database”
- 3.1.5.Code Segment screenshots: “record the time taken for each algorithm”
- 3.1.6.Screenshot of the Normalized DB Table Structure used for this Game Option

3.2. UI:

- 3.2.1.UI screenshot when asking user to enter the inputs & answers
- 3.2.2.Explain the Validations and Exception Handling in this application.

3.3. Code Segment screenshots: unit Testing

4. Knight's tour Problem

4.1. Functionality:

- 4.1.1.Code Segment screenshots: “Choose a random starting cell”
- 4.1.2.Code Segment screenshots “8*8 chess board algorithm to identify optimum sequence of moves”

- 4.1.3.Code Segment screenshots “16*16 chess board algorithm to identify optimum sequence of moves”
- 4.1.4.Code Segment Screenshot of “save that person's name along with the correct response in the database”
- 4.1.5.Code Segment screenshots: “record the time taken for each algorithm”
- 4.1.6.
- 4.1.7.Screenshot of the Normalized DB Table Structure used for this Game Option
- 4.2. UI:
 - 4.2.1.UI screenshot when asking user to enter the inputs & answers
 - 4.2.2.Explain the Validations and Exception Handling in this application.
- 4.3. Code Segment screenshots: unit Testing

5. Sixteen queens’ puzzle

- 5.1. Functionality:
 - 5.1.1.Code Segment screenshots: “Using a Sequential program, find the maximum number of solutions”
 - 5.1.2.Code Segment screenshots: “Using a Threaded program, find the maximum number of solutions”
 - 5.1.3.Code Segment Screenshot: “save that person's name along with the correct response in the database”
 - 5.1.4.Code Segment Screenshot: “If another game player provides the same right response, indicate that the solution has already been recognized. Ask them to try again until the maximum number of solutions has been achieved”
 - 5.1.5.Code Segment Screenshot: “When all the solutions have been identified by game players, the system should clear the flag that indicate solution has already been recognized”
 - 5.1.6.Code Segment screenshots: “record the time taken for each algorithm approach”
 - 5.1.7.Screenshot of the Normalized DB Table Structure used for this Game Option
- 5.2. UI:
 - 5.2.1.UI screenshot when asking user to enter the inputs & answers
 - 5.2.2.Explain the Validations and Exception Handling in this application.
- 5.3. Code Segment screenshots: unit Testing

MARKING SCHEME

	0-25 %	26-50 %	51-75 %	76-100%
Minimum Cost Problem Application & Group Report (10 marks)	Implement Algorithm & use of appropriate Application Data Structures	Implementation of the Functionality Asked for in the Question	Unit Testing Code, User interface & Validations	Overall, understanding of the game's code-level logic based on VIVA Q&A
Minimum Cost Problem I Individual Report (10 marks)	Chart Containing the time Taken for each algorithm Technique when run the Game Individually for 15 Game Rounds	Explain the Program Logic used to solve the problem	Analyze the Complexity of the algorithms based on the Program outputs & Program logic	Comparison of the algorithmic approaches
Snake and Ladder Game Problem Application & Group Report (10 marks)	Implement Algorithm & use of appropriate Application Data Structures	Implementation of the Functionality Asked for in the Question	Unit Testing Code, User interface & Validations	Overall, understanding of the game's code-level logic based on VIVA Q&A
Snake and Ladder Game Problem Individual Report (10 marks)	Chart Containing the time Taken for each algorithm Technique when run the Game Individually for 15 Game Rounds	Explain the Program Logic used to solve the problem	Analyze the Complexity of the algorithms based on the Program outputs & Program logic	Comparison of the algorithmic approaches
Traffic simulation Problem Application & Group Report (10 marks)	Implement Algorithm & use of appropriate Application Data Structures	Implementation of the Functionality Asked for in the Question	Unit Testing Code, User interface & Validations	Overall, understanding of the game's code-level logic based on VIVA Q&A
Traffic simulation Problem Individual Report (10 marks)	Chart Containing the time Taken for each algorithm Technique when run the Game Individually for 15 Game Rounds	Explain the Program Logic used to solve the problem	Analyze the Complexity of the algorithms based on the Program outputs & Program logic	Comparison of the algorithmic approaches
Knight's tour Problem Application & Group Report (10 marks)	Implement Algorithm & use of appropriate Application Data Structures	Implementation of the Functionality Asked for in the Question	Unit Testing Code, User interface & Validations	Overall, understanding of the game's code-level logic based on VIVA Q&A
Individual Report (10 marks) Individual Report (10 marks)	Chart Containing the time Taken for each algorithm Technique when run the Game Individually for 15 Game Rounds	Explain the Program Logic used to solve the problem	Analyze the Complexity of the algorithms based on the Program outputs & Program logic	Comparison of the algorithmic approaches
Sixteen queens' puzzle Application & Group Report (10 marks)	Implement Algorithm & use of appropriate Application Data Structures	Implementation of the Functionality Asked for in the Question	Unit Testing Code, User interface & Validations	Overall, understanding of the game's code-level logic based on VIVA Q&A
Sixteen queens' puzzle Individual Report (10 marks)	Chart Containing the time Taken for each algorithm Technique when run the Game Individually for 15 Game Rounds	Explain the Program Logic used to solve the problem	Analyze the Complexity of the algorithms based on the Program outputs & Program logic	Comparison of the algorithmic approaches